ChatGPT

Prior Art for Routability-Driven PCB Placement and Congestion-Aware Detailed Routing

Executive finding

The measured behavior points to **two different failure mechanisms**, and the literature suggests they should be fixed separately rather than by making the whole placement looser.

First, the short intra-block opens are primarily a **local geometric routability problem**. Once a block has been packed and frozen as a rigid macro, no top-level macro rearrangement and no net-order campaign can create a missing escape throat between two pads or component bodies. IC routability-driven placement solved the analogous problem by estimating routing **demand versus supply** and creating whitespace specifically where demand exceeds supply, rather than uniformly inflating every spacing rule. RUDY, probabilistic routing-demand maps, RSMT-based estimators, cell inflation, and narrow-channel handling are all variations on that principle. [1]

Second, the reproducible 203/235 plateau, followed by *worse* results when the order changes but already-routed nets remain effectively fixed, is characteristic of an **order-dependent routing state**. Negotiated-congestion routers were invented specifically to remove this first-come/first-served lock-in: routes are repeatedly ripped up and rerouted while present congestion and a persistent history cost make repeatedly contested resources less attractive. PathFinder is the canonical formulation; FastRoute, NCTU-GR, CUGR, SPRoute, TritonRoute and Freerouting implement different descendants of the same broader iterative/rip-up-and-repair idea. [2]

For your particular architecture, I would rank the changes this way:

Priority	Small change	Expected payoff	Implementation cost
A	Add a **local escape-throat / cut-capacity check** to the block packer, with spacing increased only where a net actually needs to pass	Very high	Low–moderate
A	Add **selective victim rip-up plus persistent resource-history cost** to `kicad-route`	Very high	Moderate
B	Add an incremental **RUDY/capacity overflow score** to block packing and rigid-macro arrangement	High	Low
B	Pre-route dense connector/package pins to **escape portals** before ordinary routing	Medium–high	Moderate

Priority	Small change	Expected payoff	Implementation cost
C	Put a FastRoute-like global router fully inside the placement search	Medium	High

The important distinction is that the first placement change should be made **inside the isolated block packer**, not only in the second-level macro arranger. The troublesome eFuse resistor nets are internal to the macro; once the block is rigid, their routability is already decided.

Two source caveats are worth stating explicitly. I could not verify **Horizon EDA** as an automatic PCB placement research system; its public repository is an EDA suite and exposes interactive-routing functionality, but I found no documented routability-driven automatic placer comparable with the IC literature. [3] Likewise, the public **DeepPCB** KiCad repository exposes a client/plugin, not the learned placement/routing implementation; vendor material describes reinforcement-learning-based PCB automation, but the congestion objective, placement state/action model and routing cost function are not public enough to compare algorithmically. [4] I also did not verify a primary-source "Autodesk/InstaDeep" placement algorithm or collaboration matching the wording in the question; I would treat that name as unverified rather than infer an architecture from marketing material.

Routability-driven placement

The simplest established congestion estimator is **RUDY**, Rectangular Uniform wire DensitY. For net n, with bounding box $w_n \times h_n$, estimated routed length L_n, average wire-to-wire pitch $\bar{p}$, and ℓ routing layers, Spindler and Johannes define an effective wire width $p = \bar{p}/\ell$ and uniform demand

$$d_n = \frac{L_n p}{w_n h_n}.$$

The total congestion field is the superposition of these per-net rectangle densities. The original paper permits either HPWL or RSMT length for L_n, but reports that HPWL correlated sufficiently well with routed length while being much cheaper; RUDY was almost 11× faster than the probabilistic estimator used for comparison on its test set. [5]

RUDY is especially attractive for your deterministic packing placer because it does **not** require an analytical placement framework. Nothing about the estimator itself requires gradient descent, force-directed movement or rows of standard cells. Given candidate footprint positions, you can incrementally add or subtract a net's rectangle contribution whenever one of its endpoints moves. The analytical part of the original RUDY paper was merely one way of consuming the map: its placer computed routing demand minus routing supply and moved cells away from congested areas, simultaneously reducing demand and increasing whitespace/supply. [6]

A useful distinction from the older literature is **direct versus indirect routability optimization**. Direct methods move the cells causing excessive routing demand; indirect methods allocate extra whitespace or inflate cells in congested regions so that routing supply rises. Routability-driven placers have used both: mPL combined congestion-driven movement with later whitespace allocation; ROOSTER modeled nets with rectilinear Steiner trees and a probabilistic routing model and then allocated whitespace; APlace reduced

local module density; later mixed-size placement work used cell inflation, hierarchy-aware placement and explicit narrow-channel handling. [7] This maps unusually well to your packer: you do not need to replace first-fit/backjumping; you can add a congestion term to candidate ranking and, when necessary, increase only the separation associated with the overloaded cut.

A second family is the **probabilistic route map** associated with Westra, Bartels and Groeneveld. Multi-pin nets are decomposed into two-pin connections, and demand is spread across plausible detour-free L- and Z-shaped routes rather than uniformly across an entire bounding box. The corresponding bin capacity is explicitly track-based. For a bucket of width W, height H, and routing-layer pitches p_l,

$$C_H = H \sum_{l \in H\text{-routing layers}} \frac{1}{p_l}, \qquad C_V = W \sum_{l \in V\text{-routing layers}} \frac{1}{p_l},$$

with blocked tracks removed from capacity. The estimator then compares expected usage against those capacities. [8] This is more computation than RUDY but is still vastly cheaper than detailed routing, and it has an advantage for your board: it distinguishes horizontal from vertical capacity and can account explicitly for obstructions.

Pin-density maps supplement these wire-demand models because bounding-box traffic can miss a different source of difficulty: many pins concentrated in a small region. The RUDY paper's review distinguishes pin-density estimators precisely for this reason; later routability-driven placement methods often combine net congestion with density/whitespace rather than assuming wirelength alone predicts routability. [7] For PCB footprints, pin density arguably matters even more because pin access is not an abstract grid point: large pads, through holes, mask/assembly constraints, fixed component bodies and via clearances all consume local escape geometry.

Steiner-tree demand is the next accuracy step. Instead of smearing an entire net uniformly, an RSMT or rectilinear spanning tree deposits demand along likely branches. ROOSTER used RSMT structure as part of its congestion model, while FastRoute went further by constructing **congestion-driven Steiner topologies** and shifting tree edges away from overloaded regions without increasing tree wirelength. [9] This better predicts where a bus or high-fanout net will actually cross a macro boundary, but the classical RUDY result is a warning against over-engineering the estimator: RSMT length did not buy enough accuracy there to justify replacing HPWL for the cheap inner-loop estimate. [10]

The high end of the spectrum is **global-route-in-the-loop**. FastRoute was explicitly designed to make global routing fast enough to invoke during placement: it combines congestion-driven Steiner construction, edge shifting and a congestion-sensitive maze-routing cost, substantially reducing the amount of expensive maze routing required. [11] That gives much better information than RUDY because the estimator is now solving an approximate routing problem, but it changes your placer from a few arithmetic map updates into a nested place–route optimization. For a 2–3 minute deterministic packer, I would reserve it for evaluating complete block or macro arrangements, not every candidate footprint insertion.

For rigid macros, the key is to maintain **two congestion models at two levels**. While a functional block is still being packed, include all of its internal nets and exact local obstacles. Once frozen, its internal routability should become a property of the macro, and the top-level estimator should focus on inter-macro traffic through its perimeter. A RUDY-style score works for both levels, but the first level needs an additional

hard “can this pad actually escape?” test because a low aggregate congestion score does not detect a sub-millimeter zero-capacity throat.

PCB placement papers from 2024–2026 reinforce the need to keep the PCB constraints explicit. RL_PCB formulates PCB component placement as an RL problem and reports improvements in **post-routing wirelength** over stochastic placement; its public implementation emphasizes HPWL/Euclidean wirelength and non-overlap rather than an exact capacity-aware router embedded in placement. [12] PCBAgent targets high-density industrial PCB placement and emphasizes heterogeneous design constraints and user-directed correction; its published experiments cover 17 industrial tasks, with HPWL and its PCB-specific wire metric among the reported placement measures. [13] The 2026 component-centric work explicitly calls out PCB-specific complications absent or less prominent in IC macro placement: very different component sizes, single- versus double-sided assembly, board constraints, non-overlap and component-centric proximity requirements; its discrete placement strategy deliberately positions passives around a main component and uses domain knowledge such as proximity to power pins. [14]

By contrast, Google's open Circuit Training/AlphaChip framework is an IC macro floorplanner whose public objective includes proxy wirelength, congestion and density, plus fixed-macro and macro-spacing constraints. [15] That is useful algorithmic prior art, but a direct transplant misses several PCB realities. Through-vias and through-hole structures can consume resources through most or all of the stack; vendor BGA guidance explicitly treats via configuration, layer count and plane anti-pad dimensions as escape constraints. [16] PCB placement can also use two physical sides, and a nominal “routing layer” may instead be predominantly a power or ground plane whose use for signal crossovers should be deliberately restricted. The surveyed recent PCB placement papers acknowledge board/two-sided constraints, but I did **not** verify a public PCB placer among them that explicitly optimizes preservation of power-plane continuity as part of its routability objective. That remains a gap in the public descriptions rather than something I would assume is solved. [17]

What this means for your tools — Rank A, very high payoff / low–moderate cost. Add one scalar congestion/capacity score to each first-fit candidate, preferably incremental RUDY plus a directional cut-capacity map, but make **zero-capacity local escape a hard rejection** before the normal area/locality objective. Do this in `agent-ee`'s *block packer*, while the troublesome passives can still move. Preserve your existing electrical-locality bounds lexicographically; do not replace them with congestion optimization. A full FastRoute-in-the-loop placer is a later optimization, not the first fix.

Channel reservation around dense macros

The IC notion of a **halo** is often more primitive than people assume. OpenROAD's hierarchical macro placer exposes `min_channel_size` and macro halo controls; its older macro-placement interface spaces macros by the maximum of twice the halo and the requested channel width. These are explicit user-supplied geometric constraints, not an automatically inferred formula from pin count. [18] The more principled literature treats a channel as routing supply and asks whether the cut has enough tracks for the traffic that must cross it. [19]

That leads to a simple rule you can use without adopting an IC placer. Let a prospective free channel of width s be crossed by D distinct routing branches. For usable routing layer l, let p_l be the effective

centerline track pitch and let unavailable geometry reduce the usable channel to s_l^{eff}. Then a natural discrete approximation is

$$C_{\mathrm{cut}}(s) = \sum_{l \in \mathcal{L}_{\mathrm{usable}}} \left\lfloor \frac{s_l^{\mathrm{eff}}}{p_l} \right\rfloor .$$

Require

$$C_{\mathrm{cut}}(s) \geq D + R,$$

where R is an explicit reserve, or equivalently use a utilization target $D/C_{\mathrm{cut}} < u_{\max}$. This is a PCB adaptation of the track-supply equations used by probabilistic congestion estimators and global-routing benchmarks, where edge capacity is the number of tracks and overflow is demand minus capacity. [19]

If all k usable layers have the same pitch p, a useful first-order sizing rule follows immediately:

$$s_{\mathrm{req}} \approx b + p \left\lceil \frac{D + R}{k} \right\rceil ,$$

where b represents geometry that cannot carry track centerlines because of pads, component keepouts, via anti-pads or boundary margin. This formula is a **derived engineering heuristic**, not a named standard from the cited papers; its justification is simply that channel width times track density must cover crossing demand. The important operational difference from your failed global 0.6 mm experiment is that D is evaluated **per side/per cut**, so a connector-to-mounting-hole edge that carries no required traffic does not inherit the spacing demanded by a dense eFuse cluster.

For your nominal 0.2 mm trace and 0.2 mm same-layer clearance, two adjacent different-net traces consume roughly a 0.4 mm centerline pitch in unconstrained open space. A different geometric calculation applies to squeezing one trace between two copper obstacles: with trace width w and required clearance c on both sides, the opening must satisfy

$$g \geq w + 2c,$$

so $w = c = 0.2$ mm gives a 0.6 mm **copper-to-copper throat**. That does not mean every pair of component courtyards needs a 0.6 mm gap: KiCad courtyard geometry is not itself the copper obstacle. The packer should test the actual pad, plated-hole, keepout and body constraints that matter, expanded by the applicable routing clearance. This distinction is why a global courtyard-gap parameter is too blunt for the failure you observed.

A second useful rule is to derive D from connectivity instead of raw pin count. For each candidate macro side, count the branches whose inside endpoint is in the macro and whose outside endpoint lies most naturally beyond that side. For multi-pin nets, a Steiner/MST decomposition prevents one high-fanout net from being counted as if every pin needed a separate full channel. This is the same decomposition logic used by probabilistic congestion estimators and global routers. [9] For your first-level block packer, use an even more local version: draw cuts through the candidate free corridors between neighboring parts and count how many unresolved connections must cross each cut.

Pin access makes the required whitespace **anisotropic**. A dense component may need 1.0 mm of space on one edge and essentially zero additional routing halo on another. This is consistent with narrow-channel handling in routability-driven mixed-size IC placement, where macro-induced porous/narrow spaces receive special treatment rather than assuming uniform density expansion. [20] OpenROAD's uniform halo interface should therefore be viewed as a convenient floorplanning control, not as the best algorithm for computing your PCB spacing.

PCB fanout guidance supplies useful real geometric examples. NXP's BGA guidelines show that the number of traces which can pass between adjacent land pads changes sharply with pitch, pad diameter, trace width and clearance: for 1.0 mm and 0.8 mm packages the recommended patterns can support one or, under tighter rules, two traces between adjacent pads; at 0.65 mm pitch the guidance uses staggered vias; at 0.5 mm pitch, conventional through-vias can no longer simply occupy every four-ball opening. [16] That is exactly the “supply versus demand” perspective you want at arbitrary dense PCB clusters: **do not ask whether the components are close; ask how many legal track centerlines remain through the resulting throat**.

Your four-layer stack also means layer capacity should not simply be set to four. If Top and Bottom are normal signal layers while In1/In2 are intended as GND/+3V3 planes, the conservative placement estimator should give those inner layers zero or very low signal-routing capacity except in regions where your policy explicitly permits a short crossover. That turns your actual stack into something closer to two primary routing layers plus conditional emergency capacity, rather than misleading the packer into believing a narrow channel has four independent lanes. Vendor BGA guidance illustrates why the layers are coupled: through-vias require inner-plane anti-pads and therefore consume plane geometry even when the signal itself is routed elsewhere. [16]

A particularly cheap implementation is a **side-specific macro halo derived from crossing demand**. During isolated block packing, store four values with every completed block: required free capacity on left/right/top/bottom. At top-level macro packing, those become soft directional halos. Internally, however, use the same logic between component groups rather than only at the outer block perimeter; otherwise it will not help the eFuse resistors.

What this means for your tools — Rank A, very high payoff / low cost. Replace “minimum global packing gap” with **local track-capacity constraints**. The smallest useful version is not even a congestion heatmap: for each candidate component insertion, test a handful of horizontal/vertical cuts around its pads and reject the placement when unresolved demand exceeds the number of legal 0.4 mm-pitch lanes through that throat. Only the overloaded side gets additional spacing. This directly addresses why 0.6 mm globally is infeasible while 0.26 mm locally is unroutable.

Congestion-aware detailed routing

It is useful to separate two stages in the cited IC tools. **FastRoute, NCTU-GR, CUGR and SPRoute are primarily global routers**: they decide approximate topology and layer/corridor usage against finite capacities. **TritonRoute/OpenROAD is a detailed router**: it turns guides into exact legal wires and repeatedly repairs design-rule violations. [21] Your owned KRT-derived grid router effectively combines much of both responsibility in one engine, so ideas from both stages transfer.

The fundamental prior art is **PathFinder negotiated congestion**. Instead of treating the route placed first as the permanent owner of a scarce resource, PathFinder initially allows competing signals to use the same routing resources and then iteratively raises the price of congestion. Its resource cost has three conceptual parts,

$$c_r = (b_r + h_r)p_r,$$

where b_r is the base resource cost, p_r is a present-congestion penalty and h_r is a persistent history penalty. During early iterations shared resources are relatively cheap; in later iterations the present congestion penalty increases. Resources that remain contested also accumulate history, so a repeatedly troublesome bottleneck remains expensive after its instantaneous congestion disappears. All signals are repeatedly ripped up and rerouted, eliminating dependence on a single initial net order. [22]

The **history term is the important part for your plateau**. A simple rip-up/retry router without history can oscillate: net A takes corridor X, net B displaces it; next pass A takes X back. Present congestion alone records only the current board. History tells both nets, in effect, "this corridor has caused trouble for several passes; spend some extra length elsewhere." PathFinder deliberately increases the present penalty gradually rather than making contested resources instantly impossible, giving the collection of nets a chance to negotiate rather than reproducing another fixed ordering. [23]

FastRoute adapts the same congestion-awareness earlier in the routing flow. It chooses congestion-driven Steiner topologies, shifts horizontal/vertical tree edges out of overloaded regions without increasing wirelength where possible, and uses a nonlinear congestion-sensitive cost in maze routing. Later FastRoute implementations use sequential rip-up and reroute and additional capacity/layer heuristics. [24] NCTU-GR explicitly organizes routing around simulated-evolution-based rerouting and congestion-relaxed 3-D layer assignment; NCTU-GR 2.0 adds bounded-length maze routing and collision awareness. [25]

CUGR is especially relevant to a small multilayer PCB router because it works directly in a **3-D routing graph**. It begins with fast 3-D pattern routing, then sends hard nets through a multilevel maze-routing phase: a coarse search identifies a promising routable region, followed by a fine search for a lower-cost path. Its cost model is designed to react to changing wire/via resources, and a patching stage improves downstream detailed routability. [26] You do not need CUGR's full probability model, but its architectural lesson is applicable: easy nets should be handled cheaply; scarce search effort and repeated rerouting should be concentrated on the nets whose local alternatives have been exhausted.

SPRoute shows another convergence issue: aggressively parallel independent rerouting can enter livelock, so it progressively reduces parallelism and eventually uses finer-grained routing to ensure that conflicts are resolved rather than repeatedly recreated. [27] On your single-threaded/small-board use case, the parallel machinery is not useful, but the principle is: **the latter passes should become more selective and more deterministic, not merely reshuffle all nets randomly**.

TritonRoute takes a detailed-routing "**search and repair**" approach. The public flow has separate pin-access analysis, track assignment, initial detailed routing, iterative search/repair and a DRC engine. [28] Current OpenROAD source exposes distinct DRC cost, marker cost, fixed-shape cost, marker decay and rip-up mode to its search/repair workers, demonstrating that detailed repair is an explicit cost-guided iterative phase rather than one final maze-search pass. [29] This maps closely to your use of KiCad as the arbiter: DRC

failures can be converted into local routing penalties or a reroute work list instead of being used only as a final grade.

Freerouting is also instructive because it is already your alternate backend. Its documented execution flow is **fanout → autorouter passes → optimizer passes**. [30] In the current source, autorouting enables rip-up and sets the rip-up cost proportional to the pass number, e.g. `start_ripup_costs * p_ripup_pass_no`, so later passes do not have the same economics as the first. [31] The code I inspected clearly establishes multipass rip-up/reroute and pass-dependent rip-up cost. I did **not** verify a PathFinder-equivalent persistent *per-grid-resource* historical congestion accumulator in Freerouting's current implementation, so I would not describe Freerouting's loop as identical to PathFinder.

For your router, the smallest implementation worth doing is **selective negotiated rip-up**, not a complete rewrite:

$$\mathrm{cost}(r) = \mathrm{length}(r) + \lambda_v \,\mathrm{via}(r) + \alpha_t \,\mathrm{present}(r) + \beta\, H(r),$$

where $H(r)$ is a persistent counter on a grid edge, layer transition or—if changing data structures is undesirable—grid cell. After every failed/reroute pass, increment H on resources implicated in repeated contention, while increasing α_t slowly between passes. This is directly patterned after negotiated congestion, simplified for a PCB grid. [23]

There is one critical caveat: **rerouting only the 32 open nets cannot fix an order conflict if all 203 routed nets remain hard obstacles**. An open net can discover that a routed net blocks its only viable corridor, but history cost cannot move the blocker if that blocker is never eligible for rip-up. The minimum useful victim mechanism is therefore:

1. While searching an open net, record the IDs of routed nets occupying the resources that repeatedly close its best frontier.
2. Select a small number of those blockers as victims.
3. Rip up the open net plus its victims.
4. Reroute that small set with the history-augmented cost.
5. Repeat until completion or a bounded no-improvement criterion.

This is an adaptation/inference from PathFinder's all-net negotiation and modern rip-up/search-and-repair flows, rather than a claim that the cited routers use this exact PCB-specific victim heuristic. [32] It captures most of the benefit while avoiding an all-235-net reroute every pass.

The history should ideally attach to **routing resources rather than just Cartesian cells**. A horizontal edge on Top, a vertical edge on Bottom and a via at the same x, y are different resources with different alternatives. That is also how global routing capacity is naturally represented: routing demand consumes graph edges, and overflow occurs when demand exceeds edge capacity. [33] A cell history map is acceptable as a first implementation if changing the router graph is expensive, but edge/layer history is the cleaner endpoint.

For the two plane layers, do not give them the same base cost as Top/Bottom merely because the grid can physically put a segment there. Treat them as either forbidden or **high-base-cost conditional resources**. If your board policy permits a short track on In1/In2, enable it as an expensive escape option over a bounded

distance and let the router pay for it only when the alternative is an open connection. This is a policy adaptation to your stack, not a rule extracted from the IC papers.

Finally, no negotiated router can fix a truly zero-capacity local geometry. If the eFuse block contains a throat through which the required trace cannot physically fit under the actual pad/clearance rules, endless rip-up merely confirms infeasibility. That is why placement feasibility and negotiated routing are complementary fixes.

What this means for your tools — Rank A, very high payoff / moderate cost. Turn rip-up on only for a bounded **open-net + blocker-victim work set**, add one persistent `history[layer, edge]` array, and multiply its contribution on each failed pass. Keep your existing A*/grid search and campaign machinery. Net-order campaigns can remain as an initial condition, but they should cease to be the only mechanism for escaping a bad early ownership decision.

Escape and fanout

Escape routing is prior art for exactly the case where the normal global router should **not** make the first few routing decisions independently. Dense pin fields have strong local topology constraints: if exits are chosen badly near the package, a perfectly adequate global channel can become unreachable.

The PCB-routing literature calls an important version **ordered escape routing (OER)**. In OER, pins inside an array or dense field must reach boundary locations while preserving a specified order, avoiding crossings that would make later area routing impossible. A DAC 2010 approach explicitly decomposes the problem into routability-driven pin assignment, global routing and detailed routing. [34] More recent work formulates large ordered-escape problems with multicommodity flow and capacity/order constraints, and multi-capacity formulations extend the idea to staggered/grid pin arrays and heterogeneous escape resources. [35] The essential lesson is that **exit assignment is part of routing**, not merely the first few arbitrary moves of a general maze router.

Vendor PCB practice says the same thing in less formal terms. Freescale/NXP explicitly recommends completing component fanout **before routing the nets** so the actual routing space and restrictions are known; it gives patterns including via-in-pad, moving some vias toward the outside or center of the field, and arranging vias at 45° to create two central routing channels. [36] NXP's BGA guide likewise chooses fanout topology according to ball pitch, via geometry and trace/space capability before normal routing proceeds. [16]

There is a useful elementary capacity formula behind the vendor tables. For two circular/rectangular pad obstacles whose edge-to-edge opening is g, N parallel traces of width w, with clearance c to each pad and between adjacent traces, fit only if

$$Nw + (N+1)c \le g.$$

Thus

$$N_{\max} = \left\lfloor \frac{g - c}{w + c} \right\rfloor.$$

That is a derived geometry relation; NXP's published BGA design tables provide real examples of the same phenomenon, showing one- versus two-trace escapes as pitch, land size and trace/space change. [37] It is more informative to a router than asking only whether the gap exceeds one nominal clearance value.

For a connector pin field, the most transferable OER concept is a set of **escape portals** on the perimeter of the connector or functional block. Before global routing, enumerate legal short routes from each dense pin to a small set of perimeter points. Assign portals with a cost containing stub length, preferred direction toward the other endpoint, portal/channel occupancy and an order penalty for bus crossings. Once assigned, route and lock only those short stubs; the normal global/grid router sees the portals as the new terminals.

For a one-dimensional connector carrying a bus, maintaining pin order at the escape boundary is particularly valuable: two bus nets that reverse their order in the first millimeter must cross later or change layers. Ordered escape formulations exist precisely to prevent that local decision from poisoning the global routing problem. [38] For non-bus signals, the order constraint can be softened and portal congestion becomes the main criterion.

For BGA-like fields, standard **dog-bone/fanout** behavior emerges naturally from the same portal abstraction. Outer rows can often escape directly on the surface; deeper rows need vias and other layers. NXP's guide reports that 1.0/0.8 mm fields commonly route outer rows without a via while inner positions use fanout vias; tighter pitches require staggered patterns or different via technology. [16] AMD similarly notes that package pitch and pad size determine available escape channels and that trace neck-down may be needed in the breakout region. [39]

An escape prepass is also useful even without BGAs. Your dense setting-resistor groups can be viewed as tiny pin fields: rather than requiring the main router to discover a path through a packed set of passives after all other routes are installed, the packer can demand that each electrically important pad have a legal short path to an **outside-of-cluster portal**. This is essentially pin-access analysis at PCB scale. TritonRoute's separation of pin access from track assignment and detailed search is strong IC precedent for doing this before ordinary detailed routing. [28]

The escape stage should remain short and local. It is not necessary to route a long connector bus completely in advance; doing so would recreate the same greedy problem at a different level. Its job is to consume the constrained geometry inside the pin field and hand the global router a set of terminals located in less constrained space.

What this means for your tools — Rank B, medium–high payoff / moderate cost. Add an optional **escape-portal prepass** for footprints or blocks whose pin-access density exceeds a threshold. Enumerate only short legal stubs to the footprint/block boundary, choose portal assignments by length + occupancy + order, commit those stubs, then invoke the existing router. Start with connectors and any dense cluster that has produced recurrent opens; do not apply it indiscriminately to every two-pad passive.

Benchmarks and evaluation

There is now a usable open PCB corpus, although the ecosystem is much less standardized than IC routing benchmarks.

PCBench is the most immediately applicable established dataset. Its public repository contains **164 PCB designs**, including raw and processed KiCad boards, routing-oriented JSON/PCB-RDL representations, augmentation tools and an RL environment. [40] It is routing-focused rather than a canonical industrial placement contest, but the availability of native KiCad designs makes it a better match for your toolchain than converting IC LEF/DEF benchmarks.

Especially relevant is that current **Freerouting already contains a PCBench/KiCad-10 evaluation pipeline**. Its scripts strip existing traces/vias, export Specctra DSN, preserve the original routed boards as ground truth, run routing, and use KiCad 10 DRC as an external arbiter. The repository explicitly documents conversion caveats, including copper-to-edge information omitted by the DSN path and SES re-import limitations, and therefore treats the original KiCad DRC as authoritative. [41] That pipeline is almost exactly the evaluation architecture you described and is worth copying conceptually even if you do not reuse its scripts.

A newer 2026 option is **PCBWorld/PCBWorld-Bench**. Its paper describes a KiCad-engine-grounded routing environment, two synthetic generators and **679 real open-source boards**, with completed layouts scored through eight engine-checked metrics. The agent operates through native routing/DRC operations instead of an abstract image/grid-only environment. [42] Because it is very recent, I would treat it as complementary to PCBench until its board-selection stability and long-term maintenance are clearer, but methodologically it is an excellent match for your principle that KiCad—not the router's internal geometry model—determines correctness.

PCB-Bench, published for ICLR 2026, includes 174 complete real-world projects from OSHWHub/JLCPCB material, but its stated purpose is benchmarking LLM/MLLM PCB reasoning and design understanding rather than providing a standardized autorouter-quality contest. [43] It can supply additional realistic projects, but I would not use its published LLM scores as comparison points for `kicad-route`.

Open PCB **placement** benchmarking is weaker. RL_PCB supplies code and datasets associated with its DATE 2024 experiments and uses post-routing wirelength as a central outcome. [12] PCBAgent's 17 industrial cases are valuable published evidence but are not, from the materials I verified, an openly standardized placement benchmark suite on the model of ISPD's IC contests. [44] The 2026 component-centric paper evaluates on more than nine real-world PCBs, again providing evidence for a method rather than a community-standard routing corpus. [45]

For congestion metrics, the IC global-routing contests provide the clearest language for comparison. The ISPD 2007 Global Routing Contest ranked solutions primarily by **total overflow**, with **maximum overflow** as a first tie-break and routed wirelength as another quality measure. Edge capacity represented available routing tracks; demand above that capacity generated overflow. [33] These metrics map directly to a coarse placement congestion map:

$$\mathrm{overflow}(e) = \max(0, D_e - C_e),$$

with both $\sum_e \mathrm{overflow}(e)$ and $\max_e \mathrm{overflow}(e)$ reported.

For detailed routing, **open connections and DRC legality come before aggregate wirelength**. The ISPD 2018 detailed-routing contest declared a solution invalid if any net was open, then scored legal/connected solutions using weighted short/spacing/min-area violations, via count, wirelength, guide deviation, off-

track/wrong-way routing and runtime-related factors. [46] That ordering is a good model for your PCB metric hierarchy: a shorter route with 32 unrouted nets is not “better” merely because its total wirelength is attractive.

I would therefore report every experiment in four layers:

Metric class	What to report
Feasibility	Routed nets / total nets, open connections, whether all nets are connected
Manufacturability	KiCad DRC total and counts by violation class
Routing quality	Total trace length, via count, layer usage, optional length/detour statistics
Congestion/ process	Total and maximum estimated overflow, runtime, number of rip-ups/reroutes, passes to best result

That reporting scheme combines the global-routing notion of capacity overflow with the detailed-routing requirement of zero opens/violations. [47] For your current design, keep the existing **203/235** baseline exactly as a primary result rather than collapsing it into a composite score; the 32 opens are the phenomenon the new algorithms are supposed to remove.

The most revealing ablation would compare four configurations on the **same placements and KiCad version**: current single pass; history cost without victim rip-up; victim rip-up without history; and victim rip-up plus history. Based on negotiated-congestion prior art, the middle two distinguish whether improvement comes merely from allowing displacement or from persistent avoidance of repeatedly contested resources. [23] On placement, similarly compare current packing; local escape feasibility only; RUDY/capacity score only; and both together. This lets you establish whether the cheap hard throat test already captures most of the placement benefit before investing in a more sophisticated estimator.

PCBench should be stratified rather than averaged blindly: include some 2-layer boards, several 4-layer boards, and denser/higher-layer designs, while retaining your automotive board as the primary target because public datasets may not reproduce your plane-layer policy, fixed mounting geometry, RF corridors or unusually strict component-locality constraints. Freerouting's own PCBench integration already uses a small stratified proof-of-concept spanning 2-, 4- and 6+-layer boards. [41]

The success criterion for the routing change should be stronger than “one campaign got more nets”: negotiated routing is specifically intended to reduce sensitivity to routing order. PathFinder reroutes the same collection repeatedly while costs evolve rather than relying on a lucky permutation. [23] Accordingly, report the spread across several initial net orders, even if the router itself remains deterministic once an order is supplied. A useful improvement is one that moves the median toward 235/235 **and narrows the order sensitivity**, not one that finds a single favorable permutation.

What this means for your tools — Rank A for measurement, high diagnostic payoff / very low implementation cost. Adopt **PCBench + KiCad 10 DRC** as the external regression corpus, add total/max overflow from the new coarse congestion estimator, and preserve routed-net count/open nets as the top-level score. For the first development cycle, the decisive matrix is only four router variants and four placer

variants; that is enough to tell whether selective negotiated rip-up and local route-throat checks capture the expected benefit before you invest in a full global-route-in-placement architecture.

---

1 5 6 7 9 10 cecs.uci.edu
https://www.cecs.uci.edu/~papers/date07/PAPERS/2007/DATE07/PDFFILES/08.7_1.PDF

2 22 23 32 PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs
https://janders.eecg.utoronto.ca/1387/readings/pathfinder.pdf?utm_source=chatgpt.com

3 GitHub - horizon-eda/horizon: Horizon is a free EDA package · GitHub
https://github.com/horizon-eda/horizon?utm_source=chatgpt.com

4 GitHub - instadeepai/deeppcb-kicad-plugin: DeepPCB routing plugin for KiCad · GitHub
https://github.com/instadeepai/deeppcb-kicad-plugin

8 19 Probabilistic Congestion Prediction
https://www.cs.york.ac.uk/rts/docs/SIGDA-Compendium-1994-2004/papers/2004/ispd04/pdffiles/p204.pdf

11 21 24 FastRoute: a step to integrate global routing into placement
https://scispace.com/pdf/fastroute-a-step-to-integrate-global-routing-into-placement-4nvs84tnpo.pdf?utm_source=chatgpt.com

12 RL_PCB/README.md at main · LukeVassallo/RL_PCB · GitHub
https://github.com/LukeVassallo/RL_PCB/blob/main/README.md?utm_source=chatgpt.com

13 44 Proceedings of the 30th Asia and South Pacific Design Automation Conference: PCBAgent: An Agent-based Framework for High-Density Printed Circuit Board Placement
https://dlnext.acm.org/doi/pdf/10.1145/3658617.3697736?utm_source=chatgpt.com

14 Component Centric Placement Using Deep Reinforcement Learning
https://arxiv.org/html/2602.23540v1?utm_source=chatgpt.com

15 circuit_training/README.md at main · google-research/circuit_training · GitHub
https://github.com/google-research/circuit_training/blob/main/README.md?plain=1&utm_source=chatgpt.com

16 37 AN10778 PCB layout guidelines for NXP MCUs in BGA packages
https://www.nxp.com/docs/en/application-note/AN10778.pdf

17 Component Centric Placement Using Deep Reinforcement Learning
https://arxiv.org/pdf/2602.23540?utm_source=chatgpt.com

18 Hierarchical Macro Placement — OpenROAD documentation
https://openroad.readthedocs.io/en/latest/main/src/mpl/README.html?utm_source=chatgpt.com

20 Routability-driven placement for hierarchical mixed-size circuit designs
https://dl.acm.org/doi/epdf/10.1145/2463209.2488921?utm_source=chatgpt.com

25 NCTU-GR: Efficient Simulated Evolution-Based Rerouting and Congestion-Relaxed Layer Assignment on 3-D Global Routing | IEEE Journals & Magazine | IEEE Xplore
https://ieeexplore.ieee.org/document/5703167?utm_source=chatgpt.com

26 CUGR: Detailed-Routability-Driven 3D Global Routing with Probabilistic Resource Model
https://cwpui.com/doc/c10.pdf?utm_source=chatgpt.com

27 SPRoute: A Scalable Parallel Negotiation-based Global Router | IEEE Conference Publication | IEEE Xplore
https://ieeexplore.ieee.org/document/8942105/%3Bjsessionid%3D9DEE74EA75E524265C679E05E52E952F?utm_source=chatgpt.com

28 GitHub - The-OpenROAD-Project/TritonRoute: UCSD Detailed Router · GitHub
https://github.com/the-openroad-project/tritonroute?utm_source=chatgpt.com

29 OpenROAD/src/drt/src/TritonRoute.cpp at master · The-OpenROAD-Project/OpenROAD · GitHub
https://github.com/The-OpenROAD-Project/OpenROAD/blob/master/src/drt/src/TritonRoute.cpp?utm_source=chatgpt.com

30 freerouting/docs/architecture.md at master · freerouting/freerouting · GitHub
https://github.com/freerouting/freerouting/blob/master/docs/architecture.md?utm_source=chatgpt.com

31 freerouting/src/main/java/app/freerouting/autoroute/BatchAutorouterV19.java at master · freerouting/freerouting · GitHub
https://github.com/freerouting/freerouting/blob/master/src/main/java/app/freerouting/autoroute/BatchAutorouterV19.java?utm_source=chatgpt.com

33 47 D-ISPD2007-gr-contest
https://www.ispd.cc/ispd2026/slides/2007/gr-contest.pdf?utm_source=chatgpt.com

34 38 Ordered escape routing via routability-driven pin assignment
https://dl.acm.org/doi/epdf/10.1145/1785481.1785579?utm_source=chatgpt.com

35 Proceedings of the 28th Asia and South Pacific Design Automation Conference: Efficient Global Optimization for Large Scaled Ordered Escape Routing
https://dl.acm.org/doi/epdf/10.1145/3566097.3567901?utm_source=chatgpt.com

36 Microsoft Word - AN3951_wrkng.doc
https://www.nxp.com/docs/en/application-note/AN3951.pdf

39 Layer Count Optimization - Layer Count Optimization - UG1099
https://docs.amd.com/r/en-US/ug1099-bga-device-design-rules/Layer-Count-Optimization?utm_source=chatgpt.com

40 GitHub - PCBench/PCBench · GitHub
https://github.com/PCBench/PCBench?utm_source=chatgpt.com

41 freerouting/scripts/pcbench/README.md at master · freerouting/freerouting · GitHub
https://github.com/freerouting/freerouting/blob/master/scripts/pcbench/README.md?utm_source=chatgpt.com

42 PCBWorld: A Benchmark Environmentfor Engine-Grounded PCB Design Automation
https://arxiv.org/html/2607.05915v4?utm_source=chatgpt.com

43 GitHub - digailab/PCB-Bench: Official implementation of PCB-Bench (ICLR 2026): A benchmark for evaluating LLMs on printed circuit board placement and routing reasoning. · GitHub
https://github.com/digailab/PCB-Bench?utm_source=chatgpt.com

45 Component Centric Placement Using Deep Reinforcement Learning
https://arxiv.org/pdf/2602.23540v1?utm_source=chatgpt.com

46 Proceedings of the 2018 International Symposium on Physical Design: ISPD 2018 Initial Detailed Routing Contest and Benchmarks
https://dl.acm.org/doi/epdf/10.1145/3177540.3177562?utm_source=chatgpt.com